Verifiable Encrypted Timed Signature Scheme and Its Application in Blockchains

Yuan Li, Junzuo Lai, Yi Liu, Liang Zhang, Haiyan Wang, Ke Ma, and Jiheng Zhang

***Abstract*—Verifiable Timed Signatures (VTS, CCS '20) add a verification feature to timed signatures, allowing a solver to check that a timed signature contains a valid signature before spending computational effort. VTS has been used in distributed protocols including atomic swaps and payment channels on scriptless blockchains. However, in existing VTS schemes, the solver can start solving as soon as the timed signature is received, which is problematic when solving should start only after a condition is met. In an atomic swap, for example, a malicious initiator with high-performance hardware could solve the timed signature faster than the counterparty anticipates, threatening the counterparty's funds. Increasing the time parameter mitigates this threat but prolongs lock-up periods for users, resulting in high opportunity costs. We propose Verifiable Encrypted Timed Signatures (VETS), a new cryptographic tool separating public verification from time-based extraction. Anyone can check that the encapsulated signature is valid, but no one can extract it through sequential computation alone. Extraction is possible only after an unlock key is released, and the key can be tied to an outside event or condition. We demonstrate VETS with an atomic swap protocol in which the initiator can start solving only after the counterparty publishes the refund transaction. Therefore, even with high-performance hardware, the initiator cannot extract the hidden signature before the refund transaction is published. Meanwhile, this "publish-then-solve" paradigm can reduce the initiator's opportunity costs. VETS may also be used in other settings, such as payment channels and delayed payments.**

***Index Terms*—Timed signatures, atomic swap, cryptocurrencies, blockchains.**

I. Introduction

TIMED cryptography [1], [2], [3], [4], [5], [6], [7], [8] is an important class of cryptographic primitives whose core idea lies in exploiting computational asymmetry: a generator can efficiently construct a time-lock puzzle, whereas a solver performs a sequential operation for time T to recover the encapsulated secret. This property aligns with applications requiring cryptographically enforced "time delays". This technology has found applications in areas such as fair contract signing [9], sealed-bid auctions [7], timed encryption [10].

Although timed cryptography can be used in many settings, early constructions do not support verifiability. In those constructions, a solver had to proceed a sequential computation for time T before confirming whether a puzzle was honestly generated. This limitation hindered the adoption of timed primitives in distributed systems, particularly within permissionless blockchains. To address this, Thyagarajan et al. introduced Verifiable Timed Signatures (VTS) [11]. VTS allows a solver to verify that a timed signature contains a valid signature (on message m under public key pk) before starting the sequential computation. This feature has gained traction in decentralized settings, especially for scriptless blockchains like Monero and Zcash. Since these platforms lack native on-chain time-lock support, VTS has become a core component for protocols like universal atomic swaps and scriptless payment channels.

Despite these advancements, existing VTS constructions share a common feature: the solving proces begins immediately upon receipt of the timed signature. However, certain practical scenarios require a decoupled approach. In these cases, the solver should verify the timed signature upon receipt but only initiate the sequential computation after some conditions are met.

Consider atomic swaps as an example. Suppose Alice wishes to exchange α coins on a scriptless blockchain for β coins owned by Bob on a script-enabled chain. Since the scriptless side cannot utilize Hash Time-Lock Contracts (HTLCs), Bob uses a VTS to lock the signature of Alice's refund transaction. Consequently, Alice must perform the sequential computation for time T before she can redeem her funds. To ensure atomicity, Alice's coins are typically locked for a longer duration than Bob's assets. This timing difference provides Bob with a sufficient window to claim Alice's coins. However, the actual time required for performing the sequential computation depends not only on the time parameter T but also on Alice's hardware performance. If Alice utilizes high-performance hardware, she may solve the VTS much faster than anticipated. In this scenario, she could reclaim her funds prematurely while Bob remains unable to claim his side, putting Bob's assets at risk.

One mitigation strategy [12], [13] is to select a big time parameter T under the assumption that Alice holds a high-performance

Received 17 February 2026; revised 15 June 2026; accepted 13 July 2026. Date of publication 20 July 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62525206, Grant 62472198, and Grant 62302194, in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2023B1515040020, in part by Guangzhou Basic and Applied Basic Research Foundation under Grant 2025A04J2146, and in part by the Fundamental Research Funds for the Central Universities. *(Corresponding author: Junzuo Lai.)*

Yuan Li, Junzuo Lai, Yi Liu, and Ke Ma are with the College of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: yl.ciphertext@gmail.com; laijunzuo@gmail.com; liuyi@jnu.edu.cn; make2024@stu2024.jnu.edu.cn).

Liang Zhang and Jiheng Zhang are with the Department of Industrial Engineering and Decision Analytics, Hong Kong University of Science and Technology, Hong Kong (e-mail: scottzhang@ust.hk; jiheng@ust.hk).

Haiyan Wang is with the Department of New Networks, Pengcheng Laboratory, Shenzhen 518066, China (e-mail: wanghy01@pcl.ac.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2026.3715374, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3715374

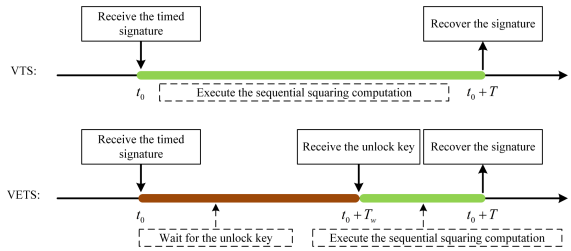

Fig. 1. A sketch of the difference between VTS and VETS.

hardware. However, a long lock time imposes non-negligible opportunity costs on ordinary users, including frozen capital and missed trading opportunities.

This observation gives rise to a natural question: *Can we design a timed signature primitive that allows the solver to verify the validity of the encapsulated signature, but commence the sequential computation for time T to recover it only after a predefined condition is satisfied?*

A. Our Contributions

In this paper, we propose a Verifiable Encrypted Timed Signature (VETS) scheme to address the aforementioned limitation. As with VTS, VETS allows a solver to verify that a received timed signature includes a valid signature σ on message m under public key pk. Unlike VTS, however, the solver must first obtain an unlocking key UK before commencing the sequential computation required to recover the signature. Fig. 1 illustrates the key differences between the VTS and VETS workflows.

The main contributions of this paper are summarized as follows:

1. We introduce *Verifiable Encrypted Timed Signatures* (VETS), a new cryptographic primitive that extends verifiable timed signatures with *conditional activation*. VETS allows a receiver to efficiently verify the existence of a valid signature, while preventing its extraction through sequential computation until the receiver obtains the unlock key.
2. We demonstrate the practicality of VETS by designing an atomic swap protocol, illustrating how VETS can be used to align the beginning of the sequential computation with an on-chain event. In other words, Alice can execute the sequential computation for redeeming her coins *iff* Bob publishes his refund transaction.
3. We comprehensively analyze the security of the VETS scheme. Hereafter, we analyze the security of our atomic swap protocol under the Universal Composability (UC) framework, and our performance evaluation shows that the proposed scheme incurs only modest overhead.
4. Beyond atomic swaps, we believe that such a primitive would benefit a range of applications requiring event-triggered time-locks, such as conditional delayed payments on anonymous cryptocurrencies, and scriptless payment channels with unlimited lifespan.

B. Organization

Section II presents multiple research works about the timed primitives and the protocols for atomic swap transactions. Section III introduces the preliminaries relevant to this work, including time-lock puzzles and verifiable timed signatures. Section IV presents the proposed verifiable encrypted timed signature scheme, including its formal definition and concrete construction. Section V describes our atomic swap protocol, including the system model and the details of the construction. Section VI evaluates the performance of the VETS scheme and the atomic swap protocol. Section VII introduces two potential applications: payment channels and delay payments on scriptless blockchains. Finally, Section VIII concludes the paper. The appendix provides supplementary material, including the ideal functionality of our atomic swap protocol and the corresponding UC proof.

II. Related Works

A. Time-Based Cryptography and Verifiable Timed Signatures

Time-based cryptography is a class of cryptographic primitives designed to enable senders to securely transmit information into the future. This encapsulated secret can only be disclosed by performing a sequential computation for time T. Representative primitives in this domain include time-lock puzzles [1], [4], [6], [7], [8], [14], [15], [16], [17], [18], timed commitments [3], [9], [19], [20], and timed signatures [11], [21], [22], [23], [24].

Verifiable Timed Signatures (VTS) [11] extend digital signatures with temporal constraints, enabling time-delayed verification and extraction. A key advantage of VTS is that both signature generation and verification require no trusted third parties, thereby enhancing decentralization and security. In VTS schemes, the prover performs $(t+1)$-out-of-n secret sharing on the signature. Through cut-and-choose techniques, the prover reveals t of these shares. The verifier then checks the correctness of the opened shares and validates the corresponding time-lock puzzles using the random coins. Upon successful verification, the verifier is assured that at least one valid share exists among the remaining unopened puzzles, enabling future reconstruction of a valid signature. Building upon this framework, researchers have proposed various verifiable timed cryptographic schemes, such as Verifiable Timed Linkable Ring Signatures (VTLRS) [24] and Verifiable Timed Discrete logarithm (VTD) [13]. However, these schemes share a common characteristic: the solver can commence sequential computation immediately upon receipt, with no mechanism to defer solving until a specified condition is satisfied. As discussed before, this proves problematic in scenarios such as atomic swaps, where premature solving by a party with high-performance hardware can compromise the security of the counterparty.

B. Atomic Swap Protocols

The concept of atomic swaps was first introduced by Nolan[1], with the fundamental objective that both parties either successfully obtain each other's tokens or safely reclaim their own

[1]https://en.bitcoin.it/wiki/Atomic_swap

assets. This property captures the essential security requirement of cross-chain exchanges: no participant should incur asset loss during protocol execution. Prior to atomic swaps, cross-chain exchanges relied on trusted intermediaries such as centralized exchanges [25], imposing additional trust assumptions and potential security risks.

Hash Time-Locked Contracts (HTLCs) [25] emerged as the foundational mechanism for trustless atomic swaps, enabling cross-chain exchanges without reliance on third parties. Nevertheless, HTLCs have not become a universal solution. On the one hand, HTLCs require specific scripting capabilities (e.g., hash verification, absolute or relative timelocks) that not all blockchains provide; on the other hand, HTLCs incur significant on-chain costs due to their multi-step transaction structure. To address these limitations, Universal Atomic Swap (UAS) [13] combined adaptor signatures with Verifiable Timed Discrete Logarithm (VTD) functions, introducing the first atomic swap protocol compatible with virtually any blockchain. UAS depends only on minimal on-chain functionality, (*i.e.,* signature verification), making it applicable to the vast majority of blockchain systems. However, as discussed in the introduction, UAS cannot prevent an initiator with superior computational resources from solving the time-lock puzzle prematurely, unless choosing a big time parameter T. Choosing such a parameter T causes high opportunity costs to ordinary users.

Recently, PipeSwap [12] identified a critical security vulnerability in UAS known as the *double-claim attack*. This vulnerability arises from differing transaction confirmation times Δ across blockchains. During the swap phase, a malicious participant can submit a competing transaction with a higher fee within the Δ-time window following an honest user's broadcast, enabling the adversary to front-run the honest transaction and violate atomicity. PipeSwap mitigates this attack by redesigning the transaction structure. However, PipeSwap still cannot prevent an initiator with high-performance computing resources from prematurely solving the time-lock puzzle to reclaim her own assets before the counterparty can safely refund, thereby violating atomicity. As with UAS, selecting a large time parameter T would cause high opportunity costs to ordinary users.

The VETS scheme proposed in this paper remains compatible with both the UAS and PipeSwap protocols. For ease of presentation, we describe our atomic swap protocol by integrating VETS into the UAS framework; however, the proposed scheme can be equally applied to PipeSwap.

III. Preliminaries

Following the definitions in the existing works [26], we use $1^\lambda(\lambda \in \mathbb{N})$ to represent the security parameter. It is implicitly provided as an input to each function, and all algorithms operate in polynomial time in the security parameter 1^λ. Besides, $x \xleftarrow{\$} \mathcal{X}$ means the variable x is uniformly chosen from the set $\mathcal{X}$. Furthermore, we use $Y \leftarrow \mathbf{A}(y)$ to represent an outcome Y is computed by a probabilistic polynomial time (PPT) algorithm $\mathbf{A}$ with an input y, and $Y := \mathbf{A}(y)$ to represent an outcome Y is computed by a deterministic polynomial time (DPT) algorithm $\mathbf{A}$ with an input y.

A. Time-Lock Puzzles

Linearly Homomorphic Time-Lock Puzzle scheme (LHTLP) [8] can generate a time-lock puzzle with Linearly homomorphic property, which can be force-open by a sequential computation for time T. The LHTLP scheme includes the following four algorithms: (1) $pp_{\text{TLP}} \leftarrow \mathsf{LHTLP.PSetup}(1^\lambda, T)$: a PPT algorithm, which inputs a security parameter 1^λ and a time hardness parameter T outputs a public parameter pp_{TLP}. (2) $Z \leftarrow \mathsf{LHTLP.PGen}(pp_{\text{TLP}}, s)$: a PPT algorithm, which inputs a public parameter pp_{TLP} and a solution $s \in S$, outputs a time-lock puzzle Z. (3) $s \leftarrow \mathsf{LHTLP.PSolve}(pp_{TLP}, Z)$: a DPT algorithm, which inputs a public parameter pp_{TLP} and a puzzle Z, outputs a solution s. (4) $\hat{Z} \leftarrow \mathsf{LHTLP.PEval}(c, pp_{\text{TLP}}, \{Z_i\}_{i\in[1,n]})$: a PPT algorithm, which inputs a circuit $c \in \mathcal{C}_\lambda$ and public parameter pp_{TLP} as well as a set of n puzzles $\{Z_i\}_{i\in[1,n]}$, outputs a puzzle $\hat{Z}$. Informally, for every Parallel Random Access Machines (PRAM) adversary $\mathcal{A}$ with running time $\leq T^\varepsilon(\lambda)$, and for every pair of solutions $(s_0; s_1) \in \{0,1\}^2$, the probability to distinguish the puzzle Z generated using solution s_0 from the puzzle generated using solution s_1 (with the time hardness parameter T) is negligible. A secure puzzle generated by the LHTLP scheme needs to satisfy the following properties:

Definition 1 (Correctness): A time-lock puzzle scheme LHTLP including the three algorithms $(\mathsf{PSetup}, \mathsf{PGen}, \mathsf{PSolve})$ is correct if for any polynomial parameter T and any security parameter λ the following condition holds:

$$s = \mathsf{PSolve}(\mathsf{PGen}(T, s)), \forall s \in \{0,1\}^*.$$

Definition 2 (Security): For any polynomial-size adversary $\mathcal{A}$ with a polynomial function $\hat{\mathbf{T}}(\cdot) \leqslant \mathbf{T}(\cdot)$, the LHTLP scheme is secure if the following equation is satisfied:

$$\Pr\left[b \leftarrow \mathcal{A}(pp_{\text{TLP}}, Z) \middle| \begin{matrix} b \leftarrow_{\$} \{0,1\}, \\ Z \leftarrow \mathsf{PGen}(\mathbf{T}(\lambda)) \end{matrix}\right] \leqslant 1/2 + \mathtt{negl}(\lambda).$$

We refer reader to read [8] for more formal details.

B. Verifiable Timed Signatures

Verifiable Timed Signature (VTS) protocol [11] is a scheme in which the signer commits to a signature σ of a message m and shares it with a receiver. After a given amount of time T has passed, the receiver can recover the signature σ from the timed signature by himself. The VTS protocol consists of five algorithms: $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{Verify}, \mathsf{Open}, \mathsf{ForceOp})$. (1) $pp \leftarrow \mathsf{VTS.Setup}(\lambda, T)$: a PPT algorithm, which inputs a security parameter λ and a time hardness parameter T, outputs a public parameter $pp := (pp_{\text{TLP},crs})$; (2) $(C, \pi) \leftarrow \mathsf{VTS.Commit}(pp, \sigma)$: a PPT algorithm, which inputs the public parameter pp and a signature σ, outputs a timed signature (C, π). (C, π) includes n time-lock puzzles $C_i = (u_i, v_i)$ and their corresponding proofs π_i; (3) $\{0,1\} \leftarrow \mathsf{VTS.Verify}(C, \pi)$: a DPT algorithm, which inputs the timed signature (C, π), output a bit b. If $b = 1$, it means that the timed signature is well-formed. Otherwise, (C, π) is an invalid timed signature; (4) $\sigma \leftarrow \mathsf{VTS.Open}$: a DPT algorithm outputs the signature σ; (5) $\sigma \leftarrow \mathsf{VTS.ForceOp}(C)$: a DPT algorithm, which inputs the timed signature C, outputs the signature σ. A secure VTS protocol satisfies the following

properties: (1) *Soundness.* The verifier is confident that, given a timed signature C, the algorithm $\mathsf{VTS.ForceOp}$ can generate the valid signature σ of the message m after time T. (2) *Privacy.* All Parallel Random Access Machine (PRAM) algorithms with running time at most t (where $t < T$) can, with at most a negligible probability, successfully extract σ from (C, π).

C. ECDSA Signature

Since our concrete construction is instantiated with ECDSA, we briefly recall the scheme. Let $\mathbb{G}_1$ be a cyclic group of prime order q with generator g_1, and let $H : \{0,1\}^* \to \mathbb{Z}_q$ be a cryptographic hash function. The ECDSA scheme $\mathsf{SIG} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ consists of the following algorithms.

- $\mathsf{KeyGen}(1^\lambda)$: Sample $sk \xleftarrow{\$} \mathbb{Z}_q^*$ and set the public key $pk := g_1^{sk}$ (also denoted h in the construction). Output the key pair (pk, sk).
- $\mathsf{Sign}(sk, m)$: Compute $c := H(m)$, sample a nonce $k \xleftarrow{\$} \mathbb{Z}_q^*$, and set $R := g_1^k = (\hat{x}, \hat{y})$ and $r := \hat{x} \bmod q$. Then compute $s := k^{-1}(c + r \cdot sk) \bmod q$. Output the signature $\sigma := (r, s)$.
- $\mathsf{Verify}(pk, m, \sigma)$: Parse $\sigma = (r, s)$, compute $c := H(m)$, and set $(\hat{x}, \hat{y}) := (g_1^c \cdot pk^r)^{s^{-1}}$. Output 1 if $\hat{x} = r \bmod q$, and 0 otherwise.

IV. Verifiable Encrypted Timed Signature

We now introduce the Verifiable Encrypted Timed Signature (VETS) protocol based on the Verifiable Timed Signature (VTS) protocol [11]. In the VTS protocol, any user who receives a correctly timed signature can recover the committed signature σ by performing the sequential computation for time T. Once a user receives the timed signature, he can immediately execute the $\mathsf{VTS.ForceOp}$ algorithm to start solving. Therefore, we aim to achieve the following property: after receiving the timed signature, a user can only verify that it commits to a valid signature σ corresponding to a message m, but cannot immediately solve it (even if the user attempts to solve it right away, he cannot recover the committed signature σ). The user must first receive the unlock key UK corresponding to the timed signature, and then, recover the committed signature σ by performing the sequential computation for time T.

A. VETS Definition

Definition 3 (VETS protocol): A VETS protocol for a signature scheme $\mathsf{SIG} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ can be written as a tuple of five algorithms which are shown as follows:

1. $\mathsf{VETS.Setup}(1^\lambda, T) \to (pp, sp)$: A probabilistic algorithm that takes a security parameter 1^λ and a time hardness T, and outputs a public parameter pp and a signing parameter sp. The signing parameter sp contains a uniformly chosen $y \xleftarrow{\$} \mathbb{Z}_N^*$ together with its derived shares $\{y_i\}$; the value y also serves as the unlock key, denoted $UK := y$.
2. $\mathsf{VETS.Commit}(pp, \sigma, pk, sp) \to (C, \pi)$: A probabilistic algorithm that takes the public parameter pp, a signature σ, a public key pk, and the signing parameter sp, and outputs a timed signature C and a proof π.
3. $\mathsf{VETS.Verify}(pp, C, \pi, m) \to 0/1$: A deterministic algorithm, which inputs the public parameter pp, the timed signature C, the corresponding proof π and a message m, outputs 0 or 1. If C is correctly generated and it locks a valid signature σ of the message m for the public parameter pp, the algorithm returns 1; Otherwise, returns 0.
4. $\mathsf{VETS.Open} \to \sigma$: A deterministic algorithm outputs a signature σ.
5. $\mathsf{VETS.ForceOp}(C, UK) \to \sigma$: A deterministic algorithm that takes a timed signature C and the unlock key UK, and outputs the valid signature σ.

Following the definitions in [11], the security requirements for the VETS protocol include: (1) **Soundness:** Given a timed signature C and the corresponding unlock key UK, the user should be confident that the $\mathsf{ForceOp}$ algorithm will recover the valid signature σ of the message m by performing a sequential computation with time T. (2) **Privacy:** Any Parallel Random Access Machine (PRAM) algorithms with a running time t should have only a negligible probability of successfully extracting the valid signature σ from the timed signature C and the proof π, unless he obtains the unlock key UK and $t \geqslant T$. These requirements can be formally defined as follows:

Definition 4 (Soundness): A VETS scheme including the above algorithms is sound for a digital signature scheme $\mathtt{SIG} := (\mathtt{KGen}, \mathtt{Sign}, \mathtt{Vrfy})$, if for all probabilistic polynomial time (PPT) adversary $\mathcal{A}$ and the messages m, the following equation holds:

$$\Pr\left[\begin{array}{l|r} b_0 = 1 \\ \wedge \\ b_1 = 0 & \begin{array}{r} (pk, m, C, \pi, T) \leftarrow \mathcal{A}(1^\lambda) \\ \sigma \leftarrow \mathtt{VETS.ForceOp}(pp, C, UK) \\ b_0 := \mathtt{VETS.Verify}(pp, m, C, \pi) \\ b_1 := \mathtt{SIG.Vrfy}(pk, m, \sigma) \end{array} \end{array}\right] \leq \mathtt{negl}(\lambda).$$

Definition 5 (Privacy): A VETS scheme including the above algorithms is private for a digital signature scheme $\mathtt{SIG} := (\mathtt{KGen}, \mathtt{Sign}, \mathtt{Vrfy})$, if for any $\mathcal{PPT}$ simulator $\mathcal{S}$ and a polynomial $\hat{T}$ such that for all polynomials $T > \hat{T}$, all PRAM adversaries $\mathcal{A}$ whose running time is at most $t < T$, all messages $m \in \{0,1\}^*$, and all $\lambda \in \mathbb{N}$ it holds: unnumbered equation shown at the bottom of this page.

$$\left|\begin{array}{l} \Pr\left[\mathcal{A}(pk, m, C, \pi) = 1 \,\middle|\, \begin{array}{r} (pk, sk) \leftarrow \mathtt{SIG.KGen}(1^\lambda) \\ \sigma \leftarrow \mathtt{SIG.Sign}(sk, m) \\ (C, \pi) \leftarrow \mathtt{VETS.Commit}(pp, \sigma, pk, sp) \end{array}\right] \\ -\Pr\left[\mathcal{A}(pk, m, C, \pi) = 1 \,\middle|\, \begin{array}{r} (pk, sk) \leftarrow \mathtt{SIG.KGen}(1^\lambda) \\ (m, \sigma, C, \pi) \leftarrow \mathcal{S}(pk, T) \end{array}\right] \end{array}\right| \leq \mathtt{negl}(\lambda).$$

B. Design Rationale

We design VETS on top of VTS following a one-time-pad idea: each time-lock puzzle is additionally locked with a key so that it cannot be solved until that key is released. However, a VTS timed signature contains n puzzles, so choosing the locking keys independently would yield $O(n)$ unlock keys, which are cumbersome to distribute. To avoid this, we instead derive the n per-puzzle keys $\{y_i\}$ from a *single* unlock key UK via Shamir secret sharing, so that releasing UK alone suffices. The key benefit is that, since the signer already reveals t of these shares during verification, once the receiver obtains UK he can recompute the remaining $(n-t)$ shares by himself and then solve the remaining puzzles exactly as in VTS.

C. VETS Construction

We set the secret sharing parameter to n and define the threshold as $(\frac{n}{2}+1)$. The size of the signature σ is set to $|\sigma| = \lambda$. Let $\mathcal{H}'$ denote a secure hash function $\mathcal{H}' : \{0,1\}^* \to I \subset \{1,2,\ldots,n\}$, where $|I| = t = \frac{n}{2}$. Meanwhile, we set the RSA modulus $N = p' \cdot q'$, where p' and q' are two prime numbers. We simplify the assumption that the ForceOp algorithm is capable of solving approximately $(n-t)$ puzzles simultaneously. Due to space constraints, we present only the construction based on ECDSA. However, our approach is not limited to this setting. As with VTS, VETS is compatible with multiple signature schemes, including BLS and Schnorr signatures.

VETS.Setup: With the input $(1^\lambda, T)$, this algorithm proceeds as follows.

1. Compute $crs \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$.
2. Compute $pp_{\mathrm{TLP}} \leftarrow \mathsf{LHTLP.PSetup}(1^\lambda, T)$.
3. Select $y \in \mathbb{Z}_N^*$ and t random elements $\{a_j : a_j \in \mathbb{Z}_N^*, j \in [t]\}$, compute $Y := g^y$ and generate a t-degree polynomial function $f(x) = y + a_1 \cdot x + \cdots + a_t \cdot x^t (\mathrm{mod} N)$, where g is a generator of a cyclic group $\mathbb{G}$ with order N.
4. For each $i \in [1,n] \cap \mathbb{N}$, compute $y_i := f(i)(\mathrm{mod} N)$ and $Y_i := g^{f(i)}$.
5. Output $pp := (crs, pp_{TLP}, Y, \{Y_i\}_{i\in[1,n]\cap\mathbb{N}}, g, g_1)$ and $sp := (y, \{y_i\}_{i\in[1,n]\cap\mathbb{N}})$, where y also serves as the unlock key, *i.e.,* $UK := y$.

VETS.Commit: with the input (pp, σ, pk, sp), this algorithm proceeds as follows.

1. Parse $\sigma = (r,s), pp := (crs, pp_{\mathrm{TLP}}, Y, \{Y_i\}, g, g_1), sp := (y, \{y_i\})$, where σ is a valid ECDSA signature.
2. Let $R := (\hat{x}, \hat{y}) = (g_1^c \cdot h^r)^{s^{-1}}$ and $B = g_1^c \cdot h^r$, where $c = \mathcal{H}(\mathtt{Tx})$.
3. For each $i \in [1,t]$, randomly choose a uniform element $s_i \leftarrow \mathbb{Z}_q$ and set $R_i := B^{s_i}$.
4. For each $i \in [t+1, n]$, calculate the following ($l_i(x)$ is the i^{th} Lagrange polynomial function): $s_i = (s^{-1} - \sum_{j=1}^{t} s_j \cdot l_j(0)) \cdot l_i(0)^{-1} \pmod q$, $R_i = (\frac{R}{\prod_{j=1}^{t} R_j^{l_j(0)}})^{l_i(0)^{-1}}$.
5. For $i \in [n]$, generate timed puzzles with corresponding range proofs and lock those puzzles with the element y_i as follows.
   1) Generate a time-lock puzzle (u_i, v_i) by executing $\mathsf{LHTLP.PGen}(pp_{\mathrm{TLP}}, s_i)$.
   2) Compute $\hat{u}_i := (u_i \cdot y_i) \bmod N$, and let $Z_i := (\hat{u}_i, v_i)$.
   3) Generate $\pi_i \leftarrow \mathsf{NIZK.Prove}(crs, (Z_i, 0, 2^\lambda, T), (s_i, r_i, y_i), Y_i)$.
6. Calculate $I \leftarrow \mathcal{H}'(pk, r, R, Y, \{R_i, Z_i, \pi_i, Y_i\}_{i\in[1,n]})$, where $|I| := t$.
7. Output a timed signature $C := (r, R, (Z_1, \ldots, Z_n), T)$, $\pi := (\{R_i, \pi_i\}_{i\in[1,n]\cap\mathbb{N}}, I, \{s_j, r_j, y_j\}_{j\in I})$.

VETS.Verify: with the input (pp, m, pk, C, π), this algorithm proceeds as follows.

1. Parse the timed signature $C := (r, R, (Z_1, \ldots, Z_n), T)$, $\pi := (\{R_i, \pi_i\}, I, \{s_j, r_j, y_j\}_{j\in I})$ and $pp := (crs, pp_{TLP}, Y, \{Y_i\}, g, g_1)$.
2. If all of the following conditions are satisfied, return 1. Otherwise return 0.
   1) It holds that $\hat{x} = r \bmod q$ where $(\hat{x}, \hat{y}) := R$.
   2) For all $j \in I$ and a $i \in [n]/I, \prod_{j\in I} R_j^{l_j(0)} \cdot R_i^{l_i(0)} = R$ and $\prod_{j\in I} Y_j^{l_j(0)} \cdot Y_i^{l_i(0)} = Y$.
   3) For each index $i \in [n]$, execute the algorithm $\mathsf{NIZK.Verify}(crs, (Z_i, 0, 2^\lambda, T), Y_i, \pi_i) = 1$ and verify whether $\mathsf{GCD}(\hat{u}_i, N) := 1$.
   4) For each $j \in I$, $(u_j, v_j) = \mathsf{LHTLP.PGen}(pp_{\mathrm{TLP}}, s_j; r_j)$, $u_j = (\hat{u}_j \cdot y_j^{-1}) \bmod N$, $R_j = (g_1^c \cdot h^r)^{s_j}$ and $Y_j = g^{y_j}$.
   5) $I = \mathcal{H}'(pk, r, R, Y, \{R_i, Z_i, \pi_i, Y_i\}_{i\in[n]})$.

VETS.Open: This algorithm outputs σ.

VETS.ForceOp: with the input (C, UK), this algorithm proceeds as follows.

1. Once the unlock key $UK := y$ is acquired, the receiver can use y and $\{y_i\}_{i\in I}$ to reconstruct the original t-degree polynomial function $f(x)$ via Lagrange interpolation. Then, he can compute the remaining shares $y_j := f(j)(\mathrm{mod} N)$ where $j \in [n] \setminus I$.
2. Since t puzzles are opened in the verify algorithm, the ForceOp algorithm only needs to solve one of the remaining $(n-t)$ puzzles by doing the following.
   – Compute $u_i := \hat{u}_i \cdot y_i^{-1} (\mathrm{mod} N)$.
   – Compute $s_i \leftarrow \mathsf{LHTLP.Solve}(pp_{\mathrm{TLP}}, (u_i, v_i))$.
3. Output $(r, s := \sum_{j\in I\cup\{i\}} s_j \cdot l_j(0))$.

D. Security Analysis

Building on the security-analysis framework in [11], we analyze the soundness and privacy of our VETS scheme. We let $|\sigma| = \lambda$ be the max number of bits of σ, and $\mathcal{H}' : \{0,1\}^* \to I \subset [n]$ be a hash function with $|I| = t$ modeled as a random oracle.

Theorem 1: (Soundness) Suppose LHTLP scheme is a secure time-lock puzzle scheme with perfect correctness, then the VETS protocol for ECDSA signature satisfies soundness property as illustrated in Definition 4.

Proof: Assume that there exists an efficient adversary $\mathcal{A}$ that can break the soundness of VETS by generating an invalid timed signature $C^* = (Z_1^*, \ldots, Z_n^*)$ and a set $\{Y_1^*, \ldots, Y_n^*\}$ that can satisfy the verification algorithm. Otherwise, by interpolating the set $\{\sigma_j\}$ (where each $j \in I$ satisfies the verification algorithm)

and one share σ_i, one can recover a valid signature σ on the message m. Further observe that all puzzles $(Z_1^*, \ldots, Z_n^*)$ are well-formed, *i.e.,* the probability of the tuple (Z_i^*, Y_i^*) generated with different parameters r_i and y_i is negligible. Following the literature [11], for the puzzles $(Z_1^*, \ldots, Z_n^*)$, the verifier can randomly select an index set I^* and verify whether the puzzles Z_i^* corresponding to these indices satisfy the verification algorithm. To ensure that the verifier believes such timed signature C is valid, the adversary $\mathcal{A}$ must correctly guess the specific value of the set $I^* = I$ before the execution, *i.e.,* $\mathcal{A}$ correctly guesses a random n-bit binary string uniformly chosen from the set of strings with exactly $n/2$-many 0's. The probability of this case is $\frac{(n/2!)^2}{n!}$. To be noted that the above statement holds based on the simulation-sound NIZK proof, even if there exist a polynomial number of simulated proofs. Hence, if the construction of the range NIZK proof is a simulation-sound scheme, it also results in our VETS protocol with the property of soundness. □

Theorem 2: (Privacy) Suppose LHTLP scheme is a secure time-lock puzzle scheme with perfect correctness, then the VETS protocol for ECDSA signature satisfies privacy property as illustrated in Definition 5.

Proof: We prove that the VETS protocol preserves privacy when the adversary's computational depth is bounded by T^{ε} (where $\varepsilon \leq 1$) and the adversary does not possess the unlocking key.

Hybrid $\mathcal{H}_0$: This is the original execution of the VETS protocol.

Hybrid $\mathcal{H}_1$: Hybrid $\mathcal{H}_1$ is identical to hybrid $\mathcal{H}_0$, except that we employ a lazy sampling functionality to simulate the random oracle. Through lazy sampling, a set I^* (where $|I^*| = t$) can be generated in advance. When the cut-and-choose instance queries the random oracle, it outputs the set I^*. The above modification is purely syntactic and does not affect the overall distribution.

Hybrid $\mathcal{H}_2$: In this hybrid, we sample a simulated common reference string crs. Due to the zero-knowledge property of $(\mathsf{NIZK.Setup}, \mathsf{NIZK.Prove}, \mathsf{NIZK.Verify})$, this change is computationally indistinguishable.

Hybrid $\mathcal{H}_3 \ldots \mathcal{H}_{3+n}$: In hybrid $\mathcal{H}_{3+i}$ (for $i \in [0, n]$), the proof π_i is generated by the simulator. By the zero-knowledge property of the NIZK, the distance between adjacent hybrids is related to the security parameter.

Hybrid $\mathcal{H}_{4+n} \ldots \mathcal{H}_{4+2n-t}$: Since the Vandermonde matrix is invertible modulo N (its determinant is coprime to N, where N is the product of two large primes) [27], the unlocking key y_i corresponding to each puzzle is uniformly distributed, unless the solver can obtain at least $(t+1)$ unlocking keys. In hybrid $\mathcal{H}_{3+i}$ (for $i \in [0, n-t]$), since y_i is uniform and independent of the adversary's view, for any observed value $\hat{u}_i$, every element $y_i^* \in \mathbb{Z}_N^*$ is equally likely to be the true y_i. Hence, the adversary's probability of correctly guessing y_i is negligible. Consequently, the indistinguishability relies on the uniform distribution of y_i.

Hybrid $\mathcal{H}_{5+2n-t} \ldots \mathcal{H}_{5+3n-2t}$: In hybrid $\mathcal{H}_{5+2n-t+i}$ (for $i \in [0, n-t]$), the puzzle corresponding to the i-th element in the complement of I^* (denoted by $\hat{I}^*$) is generated via $\mathsf{LHTLP.PGen}(pp, 0^{\lambda}; r_i)$, where a depth-bounded distinguisher is employed. The indistinguishability relies on the security of the time-lock puzzle.

Hybrid $\mathcal{H}_{6+3n-2t}$: In this hybrid, the signer operates as follows. For all $i \in I^*$, it uniformly samples s_i from $\mathbb{Z}_q$ and sets $R_i = B^{s_i} = (g_1^c \cdot h^r)^{s_i}$, then computes the corresponding puzzles according to the protocol. On the other hand, for all $i \notin I^*$, it computes:

$$R_i = \left(R / \prod_{j \in I^*} R_j^{l_j(0)} \right)^{l_i(0)^{-1}}.$$

The remaining execution proceeds unchanged. For all $i \notin I^*$, we have $\prod_{j \in I^*} R_j^{l_j(0)} \cdot \prod_{i \notin I^*} R_i^{l_i(0)} = R$, as expected. Therefore, the modification in this hybrid is purely syntactic, and the resulting distribution remains identical to that of the previous hybrid.

Hybrid $\mathcal{H}_{7+3n-2t}$: This hybrid is defined as before, except that R is sampled uniformly from the group $\mathbb{G}_q$. Note that this does not alter the distribution observed by the distinguisher.

Simulator $\mathcal{S}$: $\mathcal{S}$ is designed to be equivalent to the previous hybrid, and it should be emphasized that this security analysis does not rely on any information about the signer. This concludes our proof. □

TABLE I
SUMMARY OF NOTATIONS

Notation	Definition
$\mathcal{B}_p$	Blockchain where the subscript p denotes that party $P \in \{A, B\}$ acts as the payer.
$pk^{(*)}$	Address where the superscript $(*)$ indicates that this address belongs to blockchain $\mathcal{B}_*$.
$\overline{\mathtt{Tx}}$	The complete transaction which includes a transaction body $\mathtt{Tx}$ and a witness $\mathtt{Wit}$.
$\mathtt{MulSig}(P_0, ..., P_n)$	The multi-signature script which requires a output can be spent *iff* someone have the signatures of multi-party $\{P_0, ..., P_n\}$.
$\mathtt{OneSig}(P)$	The single signature script which requires the signature of party P to spend the output.
$\geqslant T(\mathtt{AbsTime}(T))$	The absolute timelock script so that a transaction output can be made unspendable until a specified point T in the future.

V. The Construction of Our Atomic Swap Protocol

In this section, we first present some notations and the system model. Hereafter, we outline the essential building blocks upon which our protocol relies, with their formal definitions provided in the Appendix, available online. Subsequently, we elaborate on the details of the atomic swap protocol and informal security analysis.

A. Notation

Before the illustration, Table I introduces some notations used in the rest of this paper.

B. System Model

a) Blockchains: Our scheme involves two types of blockchains: script-enabled blockchains (e.g., Bitcoin) and

scriptless blockchains (e.g., Monero). Alice (the initiator) holds tokens on the scriptless blockchain and wishes to exchange them for Bob's tokens on the script-enabled blockchain. The script-enabled blockchain supports absolute time-lock scripts, enabling Bob to lock his tokens with a deterministic unlock time, and this capability is unavailable on scriptless blockchains. We assume a synchronous network model parameterized by Δ, representing the maximum delay for a broadcast transaction to achieve confirmation on either blockchain. Formally, if a user broadcasts a valid transaction at time t, it will be confirmed no later than time $(t + \Delta)$. We further assume that adversaries can observe broadcast transactions but cannot modify, delete, or selectively delay their propagation.

b) Users and Adversary: Two users, Alice and Bob, participate in the protocol. We consider a static corruption model in which the adversary $\mathcal{A}$ corrupts at most one user before protocol execution begins. The corrupted user may arbitrarily deviate from the protocol specification, including aborting, delaying transaction broadcasts, or sending malformed messages. Meanwhile, we assume that solving the time-lock puzzle is inherently sequential and that running $\mathsf{VETS.ForceOp}$ requires a sequential computation time T. We further assume $T \geq \Delta$, i.e., the sequential computation takes at least as long as the maximum confirmation delay Δ defined above. This guarantees that Alice cannot recover the encapsulated signature faster than Bob can act on-chain.

C. Building Blocks

Digital Signature: The digital signature scheme $\mathsf{SIG} := (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ employed in this work enables the generation of a valid signature σ for a message m under the corresponding public key pk. To ensure the security of the protocol, the signature algorithm is required to satisfy the standard unforgeability property [28]. To demonstrate the generality of our construction, we assume the signature algorithm $\mathsf{SIG} \in \{\text{Schnorr}, \text{ECDSA}\}$, thereby encompassing the majority of cryptocurrencies, such as Bitcoin, Ethereum, and Ripple. The formal definition of the digital signature scheme is provided in Appendix B, available online.

Adaptor Signature: An adaptor signature scheme [29] $\mathsf{ASig} := (\mathsf{PreSign}, \mathsf{PreVrfy}, \mathsf{Ext}, \mathsf{Adapt})$ allows a user to embed a statement-witness pair $(Y, y) \in \mathcal{R}$ ($\mathcal{R}$ is a hard relation) into the signing process for a message $m \in \{0,1\}^*$. Specifically, a user holding the secret key first generates a pre-signature $\tilde{\sigma}$ for the message m. This pre-signature alone cannot pass blockchain verification; however, it can be adapted into a valid signature σ (i.e., $\mathsf{SIG.Verify}(pk, m, \sigma) = 1$) using the witness y. Moreover, the witness y can be extracted from the pre-signature $\tilde{\sigma}$ and the valid signature σ. Adaptor signatures are required to satisfy several security properties, including unforgeability, witness extractability, and pre-signature adaptability. Formal definitions are provided in Appendix B, available online.

2-Party Protocol (2 PC): Following the setting in [12], [13], our protocol requires Alice and Bob to interactively generate a joint public-secret key pair (pk, sk) as well as a joint adaptor signature $\tilde{\sigma}$. At first, Alice and Bob need to jointly generate ECDSA or Schnorr key pairs. We denote this interactive protocol by $\Gamma^{\mathsf{SIG}}_{\mathsf{JKGen}}$, where $\mathsf{SIG} \in \{\text{Schnorr}, \text{ECDSA}\}$. The joint key generation protocol $\Gamma^{\mathsf{SIG}}_{\mathsf{JKGen}}$ takes a security parameter 1^λ and a group tuple $(\mathbb{G}, g, q)$ as common inputs from both parties. Upon completion, the protocol outputs the public key pk to both parties, while Alice receives the secret key share sk_a and Bob receives the secret key share sk_b.

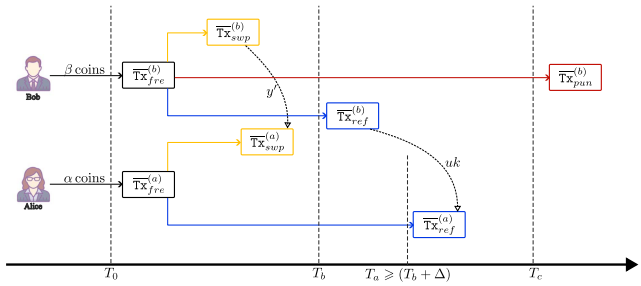

Fig. 2. The workflow of our atomic swap protocol.

Furthermore, we require Alice and Bob to jointly generate an adaptor signature $\tilde{\sigma}$ with respect to a joint public key pk for a statement Y of a hard relation R. We denote this interactive protocol by $\Gamma^{\mathsf{SIG}}_{\mathsf{ASig}}$. The protocol takes a message m and a statement Y of the hard relation R as common inputs. As private inputs, Alice and Bob provide their respective secret key shares sk_a and sk_b, where sk_a and sk_b are shares of the secret key sk corresponding to the public key pk. Upon completion, the protocol outputs a pre-signature $\tilde{\sigma}$ to both parties, satisfying $\Pi^{\mathsf{SIG}}_{\mathsf{ASig}}.\mathsf{PreVrfy}(pk, m, Y, \tilde{\sigma}) = 1$.

D. Protocol Illustration

Recall the setting of this work: Alice wishes to exchange α coins on blockchain $\mathcal{B}_a$ for β coins owned by Bob on blockchain $\mathcal{B}_b$. To ensure the atomicity of the protocol, we carefully control the release of the witness y and the unlock key UK to prevent malicious users from disrupting the protocol execution order. Specifically, even if Alice possesses high-performance computing equipment, she cannot retrieve her assets faster than Bob. Furthermore, to mitigate grief attacks, where Bob deliberately withholds the release of UK for an extended period, Alice is entitled to directly spend the coins locked by Bob after time $2T_b$, without exposing y . Fig. 2 illustrates the execution timeline of the protocol, highlighting the trigger points for three key operations: Swap, Refund, and Punish (For ease of exposition, we incorporate the description of the Punish operation into the Refund operation). Before the illustration, we assume that Alice and Bob have generated the key pairs used in the protocol by using the 2PC protocol $\Gamma^{\mathsf{SIG}}_{\mathsf{JKGen}}$.

AS.Freeze: In this phase (as shown in Fig. 3), Alice and Bob transfer their respective coins to be exchanged into two frozen addresses $pk^{(a)}_{AB}$ and $pk^{(b)}_{AB}$ that are jointly controlled by both parties. At first, each of them needs to construct the freeze transaction $\mathtt{Tx}^{(a)}_{fre}$ and $\mathtt{Tx}^{(b)}_{fre}$. Note that Alice or Bob need to satisfy the following condition to spend the output of $\mathtt{Tx}^{(b)}_{fre}$: $(\mathtt{MulSig}(A, B) \vee (\mathtt{MulSig}(A, B) \wedge$

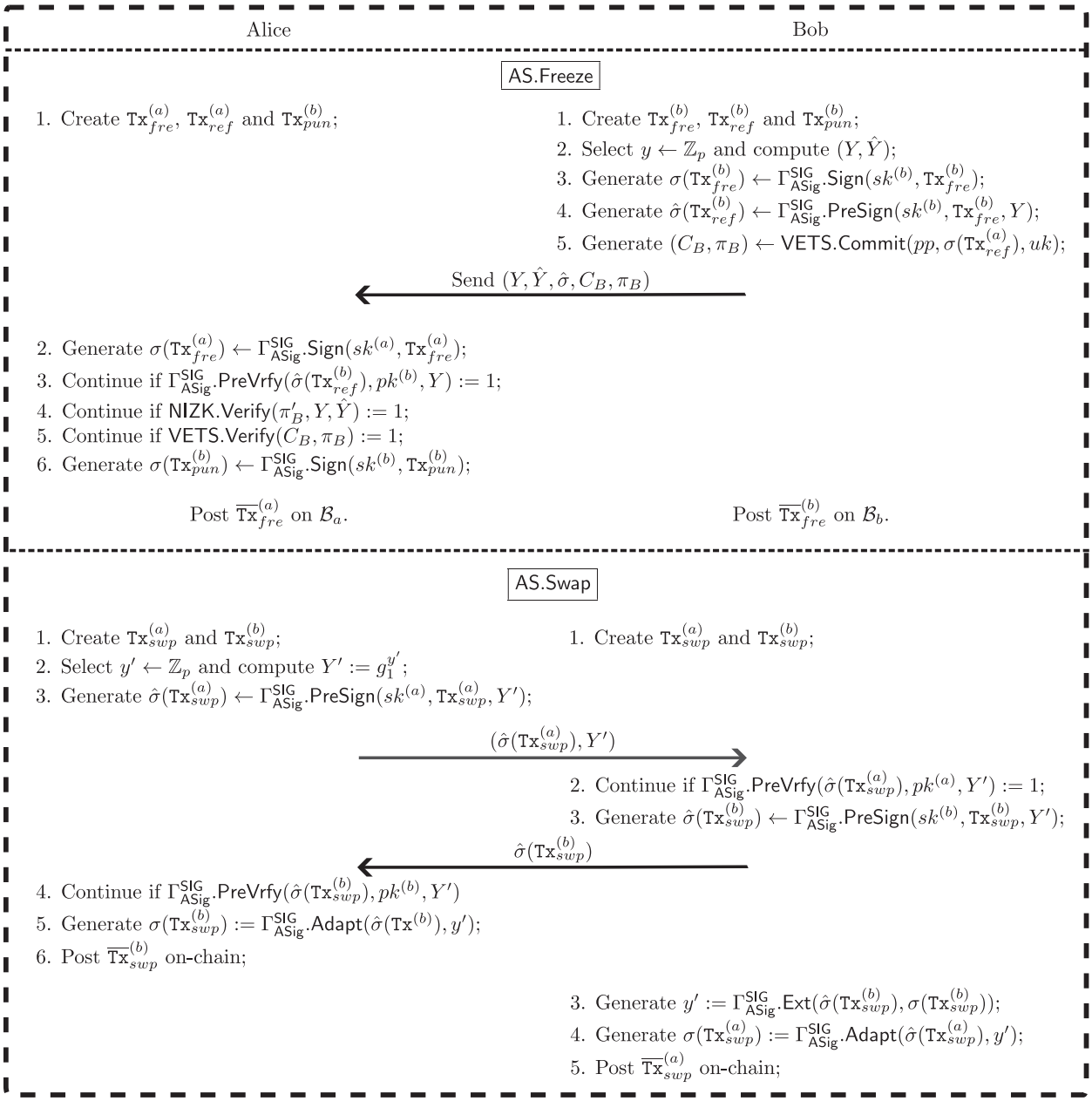

Fig. 3. Freeze and Swap phases of our atomic swap protocol.

$\mathtt{AbsTime}(T_b)) \vee (\mathtt{OneSig}(A) \wedge \mathtt{AbsTime}(T_c)))$[2]. Furthermore, they also need to construct and sign the refund transactions $\mathtt{Tx}_{ref}^{(a)}$ and $\mathtt{Tx}_{ref}^{(b)}$ used to redeem the locked coins. Specifically, Bob needs to generate a VETS by using $(C_B, \pi_B) \leftarrow \mathsf{VETS.Commit}(pp, \sigma_B(\mathtt{Tx}_{ref}^{(a)}), UK)$ where $UK := y$, and compute a statement pair $(Y, \hat{Y})$, where $Y = g_1^y$ and $\hat{Y} = g^y$. After the verification, Alice uses the statement Y to generate her adaptor signature $\hat{\sigma}_A(\mathtt{Tx}_{ref}^{(b)}) \leftarrow \mathsf{ASig.PreSign}(sk_A^{(b)}, \mathtt{Tx}_{ref}^{(b)}, Y)$ and sends it to Bob.[3] In this process, we need to generate a NIZK proof π'_B to prove such statement: $\{Y := g^y \wedge \hat{Y} := g_1^y \wedge y \in \mathbb{Z}_p\}$. Such proof can be efficiently generated by the constructions introduced in [30], [31], [32], [33], [34]. If the above steps are successfully executed, Bob first posts his freeze transaction $\mathsf{Tx}_{fre}^{(b)}$ to $\mathcal{B}_b$; only after observing Bob's on-chain freeze does Alice post her freeze transaction $\mathsf{Tx}_{fre}^{(a)}$ to $\mathcal{B}_a$.

AS.Swap : In this phase (as shown in Fig. 3), Alice and Bob need to generate the swap transactions for exchanging their locked coins. Specifically, Alice and Bob employ the 2PC

[2]When using nLockTime, the punish transaction requires joint signatures from Alice and Bob, where the nLockTime value of this transaction is set to T_c.

[3]Alice can collaborate with Bob to generate such an adaptor signature through the 2PC protocol $\Gamma_{\mathsf{ASig}}^{\mathsf{SIG}}$, contingent upon whether blockchain $\mathcal{B}_b$ supports multi-signature scripts similar to those in Bitcoin.

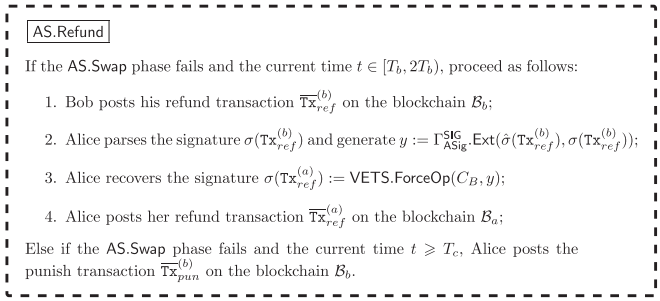

Fig. 4. Refund phase of our atomic swap protocol.

protocol to bind their respective swap transactions (*i.e.,* $\mathtt{Tx}_{swp}^{(a)}$ and $\mathtt{Tx}_{swp}^{(b)}$) together. If Alice publishes her swap transaction $\mathtt{Tx}_{swp}^{(b)}$ to claim the coins locked by Bob before time T_b, then Bob is guaranteed to be able to claim the coins locked by Alice on blockchain $\mathcal{B}_a$, thereby ensuring the atomicity of the protocol. Since this part is consistent with [13], we omit the details here for brevity.

AS.Refund: As shown in Fig. 4, if Alice fails to post the swap transaction $\overline{\mathtt{Tx}}_{swp}^{(b)}$ on-chain before time T_b, Bob posts his refund transaction $\overline{\mathtt{Tx}}_{ref}^{(b)}$ on-chain to redeem his locked coins. Upon observing the transaction $\overline{\mathtt{Tx}}_{ref}^{(b)}$, Alice extracts $UK = y$ by executing $\mathsf{ASig.Ext}(\hat{\sigma}_A, \sigma_A, Y)$ and recovers the signature $\sigma_B(\mathtt{Tx}_{ref}^{(a)}) := \mathsf{VETS.ForceOp}(C_B, y)$ after time Δ. Then, Alice posts the refund transaction $\overline{\mathtt{Tx}}_{ref}^{(a)}$ on the blockchain $\mathcal{B}_a$ to redeem her locked coins. Besides, if the swap does not occur and Bob does not retrieve his coins before time $2T_b$, Alice can spend the locked coins by posting the punish transaction $\overline{\mathtt{Tx}}_{pun}^{(b)}$ (the red line in Fig. 2).

E. Security Analysis

Theorem 3: Assuming the VETS construction satisfies the *Soundness* and *privacy* property and the 2 PC protocol is secure, the proposed atomic swap protocol satisfies atomicity: at the conclusion of the protocol, either both parties obtain the counterparty's tokens, or both parties retain their original assets.

Proof Sketch: Since the Swap phase is identical to UAS [13], we focus on the Refund phase. To protect Bob's funds, Alice's locktime T_a must satisfy $T_a \geq T_b + \Delta$, where T_b is Bob's locktime and Δ is the maximum confirmation delay. This ensures Bob can refund before Alice's lock expires.

By the *privacy* property of VETS, the signature of Alice's refund transaction $\mathtt{Tx}_{ref}^{(a)}$ remains hidden until the unlock key UK is released. Since UK is derived from Bob's refund transaction $\mathtt{Tx}_{ref}^{(b)}$, Alice cannot begin the sequential computation for recovering her refund signature, regardless of her computational resources. Additionally, to prevent Bob from indefinitely withholding his refund transaction, we introduce a deadline $T_c > (T_b + \Delta)$. If Bob fails to publish by T_c, Alice can claim Bob's funds via a penalty transaction. The formal UC proof is provided in Appendix E, available online. □

TABLE II
1024-BIT MODULAR SQUARING LATENCY SUMMARY

Platform	Latency (ns)	vs. NO.1
FPGA (Ozturk)[1]	25.2	1.0×
FPGA (Predictive Montgomery)[1]	46.0	1.83×
AMD Ryzen 5 7500F	398	15.8×

1. https://blog.janestreet.com/really-low-latency-multipliers-and-cryptographic-puzzles/

VI. PERFORMANCE ANALYSIS

In this section, we present the performance of our proposed scheme from two perspectives: the computation time corresponding to the time parameter T, and the computational overhead at each phase of our atomic swap protocol. The hardware configuration included an AMD Ryzen 5 7500F CPU operating at 5 GHz, paired with 16 GB of RAM.

A. Computation Time for Time Parameter T

As discussed previously, to prevent Alice from unlocking the timed signature before the predefined time point using high-performance computing equipment, existing solutions [12], [13] conservatively estimate Alice's computational capability and select a sufficiently large time parameter T. However, this imposes a significantly longer computation time on ordinary users. Based on the public data from [35] and the public results of the VDF competition, Table II summarizes the 1024-bit modular squaring performance across different platforms. To our knowledge, the lowest latency (25.2 nanoseconds per square operation) was achieved in the VDF competition[4] using an Field Programmable Gate Array (FPGA). Compared with the open result, our experimental platform, AMD Ryzen 5 7500F using the GMP library, achieves 398 ns per operation. As shown in the table, a VTS puzzle that requires 1 h on the FPGA (Ozturk) would take approximately 15.8 hours on our AMD Ryzen 5 7500F platform, highlighting the significant performance gap between specialized hardware and general-purpose processors for sequential computation tasks.

B. Mitigating Opportunity Cost Under Heterogeneous Hardware

In UAS and its variants, the initiator assumes the worst-case (that is, the fastest) hardware when choosing the time parameter T. As discussed before, ordinary users with low-performance hardware may spend much more time than expected. In our protocol, the start of the sequential computation is separated from the receipt of the timed signature. By doing that, Alice's effective lock duration has two parts. First, Bob's on-chain lock duration T_b, which is enforced by time-lock scripts and does not depend on computing power. Second, the time for the sequential computation, which depends on the time parameter and Alice's local hardware.

Let K be the performance ratio between normal hardware and the fastest hardware (for example, FPGA), where $K > 1$.

[4]https://www.vdfalliance.org/

TABLE III
PROBABILITIES AND COMPUTATIONAL OVERHEAD OF VTS VS. VETS UNDER DIFFERENT PARAMETER VALUES

n	t	Soundness error	VTS.Commit (ms)	VTS.Verify (ms)	VETS.Commit (ms)	VETS.Verify (ms)
20	10	5.41×10^{-6}	89.18	70.36	243.16	236.70
30	15	6.45×10^{-9}	143.94	119.96	375.32	376.74
40	20	7.25×10^{-12}	222.29	198.32	537.73	534.35
50	25	7.91×10^{-15}	333.76	312.39	721.59	732.47

Suppose Bob's assets are locked for T_b, and Alice's refund should be delayed by another Δ. In UAS, the time parameter must be chosen so that even an adversary using FPGA has to wait at least $T_b + \Delta$. Hence, a user with normal hardware will face a total lock time of:

$$T_a^{\text{UAS}} = K \cdot (T_b + \Delta).$$

In VETS, T_b is enforced only by on-chain timelocks, and it does not depend on hardware differences. Only the extra delay Δ needs sequential computation. Therefore, the total lock duration is:

$$T_a^{\text{VETS}} = T_b + K \cdot \Delta.$$

The reduction in opportunity cost for commodity users is therefore:

$$T_a^{\text{UAS}} - T_a^{\text{VETS}} = (K-1) \cdot T_b.$$

For example, with $T_b = 24$ hours and a performance ratio $K \approx 15.8$ between commodity hardware (AMD Ryzen 5 7500F) and high-performance hardware (FPGA), an ordinary user saves $(K-1) \cdot T_b = (15.8-1) \times 24 \approx 355.2$ hours of effective lock time compared with the schemes using VTS.

C. Computational Overhead

We set the parameters as $n = (20, 30, 40, 50)$, $t = n/2$ and $|N| = 1024$ bit. We use the open-source code[5] to implement the LHTLP scheme. Similar to the VTS scheme, the VETS scheme requires the signer to open t puzzles, which are randomly selected by the verifier, in order to convince the verifier that the timed signature is correctly constructed. If the verification succeeds, the verifier only needs to open one of the remaining puzzles to recover the signature. Therefore, selecting appropriate values for (n, t) is crucial to ensuring the security of the scheme. Specifically, (n, t) determine the soundness error, *i.e.,* the probability that a malicious signer constructs an invalid timed signature that nevertheless passes verification. Table III presents the corresponding probabilities, the generation time, and verification time for different values of parameter n.

For the experimental evaluation, we assume that the proposed atomic swap scheme employs ECDSA as the underlying signature algorithm. We use the open-source code[6] of [12] to implement our simulation. Table IV summarizes the computational overhead incurred by each party during the execution of our atomic swap protocol. We distinguish between the Freeze phase, where assets are locked into the blockchains, and the Swap phase, where the locked coins are exchanged. As shown in the table, most of computational costs occur in the Freeze phase, where both Alice and Bob incur approximately 640ms of computation. This cost mainly arises from the generation and verification of the timed signature and the equality proof. In contrast, the Swap phase introduces negligible overhead, requiring less than 24ms of computation for each party. The Swap phase only involves the generation and verification of adaptor signatures, as well as adapting the pre-signature into a full signature. Consequently, compared to the Freeze phase, this phase incurs lower computational overhead. These results indicate that the proposed protocol introduces only modest computational overhead for end users.

[5]https://github.com/verifiable-timed-signatures/liblhtlp
[6]https://github.com/Anqi333/pipeSwap

TABLE IV
COMPUTATIONAL OVERHEAD REQUIRED BY EACH PARTY IN THE ATOMIC SWAP PROTOCOL

Party	Freeze (ms)	Swap (ms)
Alice	588.59	22.60
Bob	593.15	23.31

VII. ADDITIONAL APPLICATIONS

This section illustrates several potential application scenarios for VETS. Due to space constraints, we focus on the conceptual framework; detailed protocol constructions and formal security analysis are deferred to future work.

A. Scriptless Payment Channels Without Lifespan Limitation

Following the system model of AuxChannel [31], we briefly sketch how VETS can be combined with Distributed Verifiable Key Escrow Services (DVKES) [36], [37] to construct bidirectional payment channels with unlimited lifespan on scriptless blockchains. In our construction, each party escrows their initial unlock key $UK^{[0]}$ with the DVKES. During off-chain payments, parties lock split transaction signatures using VETS, with unlock keys derived from the escrowed initial keys. Each unlock key $UK^{[j]}$ is computed from $UK^{[j-1]}$ using a one-way function (e.g., $x^2 \bmod N$). When a party wishes to close the channel, he publishes a commit transaction, which triggers the DVKES to release the counterparty's initial key $UK^{[0]}$. The closing party then derives the appropriate unlock key and invokes VETS.ForceOp to extract the signature required for settlement. During the ForceOp phase, if the counterparty detects that an outdated state has been used, he can post a penalty transaction to claim all funds in the channel.

Unlike VTS-based designs (for example, SleepChannel), where channels must be closed before a fixed time T, our

DVKES-based approach links key release to on-chain events. This removes the limitation of a fixed channel lifespan. Full protocol details and a security analysis are left for future work.

B. Conditional Delayed Payments on Anonymous Cryptocurrencies

We sketch how VETS enables conditional delayed payments on anonymous cryptocurrencies, providing consumer protection without sacrificing privacy.

In our construction, the consumer locks a payment signature using VETS and deposits the corresponding unlock key UK with a trusted platform (e.g., an e-commerce intermediary). The consumer then sends the VETS ciphertext to the merchant. Once the merchant successfully dispatches the goods, the platform releases UK to the merchant, who then invokes VETS.ForceOp to extract the payment signature. Crucially, this extraction requires the sequential computation for time T, during which the consumer retains the ability to reclaim their funds if he wish to return the goods.

To facilitate refunds, the merchant deposits a refund secret y with the platform using an adaptor signature mechanism. If the consumer returns the goods within time T, the platform releases y to the consumer, enabling them to complete the refund transaction before the merchant can spend the payment. This design ensures that the merchant receives payment only after dispatching goods and waiting for time T, while the consumer retains recourse throughout the refund window. A formal protocol description and security analysis of this conditional delayed payment scheme are likewise left for future work.

VIII. Conclusion

In this work, we introduced Verifiable Encrypted Timed Signatures (VETS), a new cryptographic tool. It separates public verification from time-based extraction. VETS lets anyone check whether a timed signature is valid, but the encapsulated signature can be extracted by performing the sequential computation for time T only after an unlock key is released. This key can be tied to an outside condition. Furthermore, we showed how to use VETS by designing an atomic swap protocol based on the "publish-then-solve" paradigm. This design removes the computing power weakness in earlier constructions, and it also lowers the opportunity cost for normal users. We proved its security in the Universal Composability framework, and we showed that it works well in practice through performance analysis. In addition to atomic swaps, we explained how VETS can support conditional delayed payments in anonymous cryptocurrencies and scriptless payment channels with unlimited lifetime. Full protocol details and full security analysis for these uses are left for future work.

References

[1] J. Xin and D. Papadopoulos, "Check-before-you-solve: Verifiable time-lock puzzles," in *Proc. IEEE Sym. Secur Privacy*,, San Francisco, CA, USA 2025, M. W. Blanton Enck and C. Nita-Rotaru, Eds., May 2025, pp. 2037–2056, doi: 10.1109/SP61157.2025.00053.

[2] M. ElSheikh, A. M. Youssef, and M. A. Hasan, "Self-tallying e-voting using homomorphic time-lock puzzles and ZK-SNARKs," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 2566–2581, Apr. 2025, doi: 10.1109/TNSE.2025.3550290.

[3] H. Abusalah and G. Avitabile, "Black-box timed commitments from time-lock puzzles," in *Proc. Theory Cryptography - 22nd Int. Conf.*, Milan, Italy, E. Boyle and M. Mahmoody, Eds., 2024, vol. 15366, pp. 460–493, doi: 10.1007/978-3-031-78020-2_16.

[4] R. Kumar and S. Padhye, "Chained time lock puzzle with small puzzle size," *Inf. Comput.*, vol. 304, May 2025, Art. no. 105301, doi: 10.1016/j.ic.2025.105301.

[5] A. Abadi, D. Ristea, A. Grigor, and S. J. Murdoch, "Scalable time-lock puzzle," in *Proc. 20th ACM Asia Conf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, 2025, pp. 839–855, doi: 10.1145/3708821.3733907.

[6] Y. Liu, Q. Wang, and S. Yiu, "Towards practical homomorphic time-lock puzzles: Applicability and verifiability," in *Proc. Comput. Secur. 27th Eur. Symp Res. Comput. Secur.*, Copenhagen, Denmark, V. Atluri, R. D. Pietro, C. D. Jensen, and W. Meng, Eds., 2022, vol. 13554, pp. 424–443, doi: 10.1007/978-3-031-17140-6_21.

[7] C. Freitag, I. Komargodski, R. Pass, and N. Sirkin, "Non-malleable time-lock puzzles and applications," in *Proc. Theory Cryptography 19th Int. Conf.*, Raleigh, NC, USA, K. Science Nissim and B. Waters, Eds., 2021, vol. 13044, pp. 447–479, doi: 10.1007/978-3-030-90456-2_15.

[8] G. Malavolta and S. A. K. Thyagarajan, "Homomorphic time-lock puzzles and applications," in *Proc. Adv. Cryptology - CRYPTO- 39th Ann. Int. Cryptology Conf.*, Santa Barbara, CA, USA, A. Boldyreva and D. Micciancio, Eds., 2019, vol. 11692, pp. 620–649, doi: 10.1007/978-3-030-26948-7_22.

[9] D. Boneh and M. Naor, "Timed commitments," in *Proc. Adv. Cryptology - CRYPTO 20th Ann. Int. Cryptology Conf.*, Santa Barbara, California, USA, M. Bellare, Ed., 2000, vol. 1880, pp. 236–254, doi: 10.1007/3-540-44598-6_15.

[10] L. Baird, P. Mukherjee, and R. Sinha, "i-tire: Incremental timed-release encryption or how to use timed-release encryption on blockchains?," in *Proc. ACM SIGSAC Conf. Comput. Commun Secur.*, Los Angeles, CA, USA, H. A. Yin, C. Stavrou Cremers, and E. Shi, Eds., 2022, pp. 235–248, doi: 10.1145/3548606.3560704.

[11] S. A. K. Thyagarajan, A. Bhat, G. Malavolta, N. Döttling, A. Kate, and D. Schröder, "Verifiable timed signatures made practical," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, J. Ligatti, X. Ou, J. Katz, and G. Vigna, Eds., 2020, pp. 1733–1750, doi: 10.1145/3372297.3417263.

[12] P. Ni, A. Tian, and J. Xu, "Warning! the timeout T cannot protect you from losing coins: Pipeswap: Forcing the timely release of a secret for atomic cross-chain swaps," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, M. W. Blanton Enck and C. Nita-Rotaru, Eds., 2025, pp. 1566–1583, doi: 10.1109/SP61157.2025.00245.

[13] S. A. K. Thyagarajan, G. Malavolta, and P. Moreno-Sanchez, "Universal atomic swaps: Secure exchange of coins across all blockchains," in *Proc. 43rd IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2022, pp. 1299–1316, doi: 10.1109/SP46214.2022.9833731.

[14] S. Agrawal, G. Malavolta, and T. Zhang, "Time-lock puzzles from lattices," in *Advances in Cryptology – CRYPTO 2024*, vol. 14922, L. Reyzin and D. Stebila, Eds., Cham: Springer Nature Switzerland, 2024, pp. 425–456, doi: 10.1007/978-3-031-68382-4_13.

[15] A. Abadi, "Tempora-fusion: Time-lock puzzle with efficient verifiable homomorphic linear combination," *Comput. Res. Repository*, 2024, *arXiv.2406.15070*.

[16] A. Abadi, "Verifiable homomorphic linear combinations in multi-instance time-lock puzzles," *Cryptol. ePrint Arch.*, 2024, Paper 2024/1314. [Online]. Available: https://eprint.iacr.org/2024/1314

[17] J. Dujmovic, R. Garg, and G. Malavolta, "Time-lock puzzles with efficient batch solving," in *Proc. Adv. Cryptology - EUROCRYPT- 43rd Ann. Int. Conf. Theory Appl. Cryptographic Techn.*, Zurich, Switzerland, M. Science Joye and G. Leander, Eds., 2024, vol. 14652, pp. 311–341, doi: 10.1007/978-3-031-58723-8_11.

[18] S. Srinivasan, J. Loss, G. Malavolta, K. Nayak, C. Papamanthou, and S. A. K. Thyagarajan, "Transparent batchable time-lock puzzles and applications to Byzantine consensus," in *Proc. Public-Key Cryptography 26th IACR Int. Conf. Pract. Theory Public-Key Cryptography*, Atlanta, GA, USA, A. Boldyreva and V. Kolesnikov, Eds., 2023, vol. 13940, pp. 554–584, doi: 10.1007/978-3-031-31368-4_20.

[19] S. A. K. Thyagarajan, G. Castagnos, F. Laguillaumie, and G. Malavolta, "Efficient CCA timed commitments in class groups," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Y. Kim, J. Kim, G. Vigna, and E. Shi, Eds., 2021, pp. 2663–2684, doi: 10.1145/3460120.3484773.

[20] J. Katz, J. Loss, and J. Xu, "On the security of time-lock puzzles and timed commitments," in *Proc. Theory Cryptography 18th Int./ Conf.*, Durham, NC, USA, R. Pass and K. Pietrzak, Eds., vol. 12552, 2020, pp. 390–413, doi: 10.1007/978-3-030-64381-2_14.

[21] W. Wang, J. Duan, C. Zuo, L. Wang, H. Peng, and X. Huang, "A time-bound NFT rights protocol from time interval signatures," *IEEE Trans. Dependable Secur. Comput.*, vol. 23, no. 1, pp. 209–220, Jan./Feb. 2026, doi: 10.1109/TDSC.2025.3604511.

[22] Z. Bao, D. He, Q. Feng, M. Luo, and X. Zeng, "Constant-size verifiable timed signatures from RSA group for bitcoin-based voting protocols," *IEEE Trans. Serv. Comput.*, vol. 17, no. 4, pp. 1414–1425, Jul./Aug. 2024, doi: 10.1109/TSC.2023.3347526.

[23] X. Zhou, D. He, J. Ning, M. Luo, and X. Huang, "Efficient construction of verifiable timed signatures and its application in scalable payments," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 5345–5358, 2023, doi: 10.1109/TIFS.2023.3306107.

[24] S. A. K. Thyagarajan, G. Malavolta, F. Schmid, and D. Schröder, "Verifiable timed linkable ring signatures for scalable payments for monero," in *Proc. Comput. Secur. - ESORICS 27th Europ. Symp. Res. Comput. Secur.*, Copenhagen, Denmark, V. Atluri, R. D. Pietro, C. D. Jensen, and W. Meng, Eds., 2022, vol. 13555, pp. 467–486, doi: 10.1007/978-3-031-17146-8_23.

[25] K. Narayanam, V. Ramakrishna, D. Vinayagamurthy, and S. Nishad, "Atomic cross-chain exchanges of shared assets," in *Proc. 4th ACM Conf. Adv. Financial Technol.*,Cambridge, MA, USA, M. Herlihy and N. Narula, Eds., 2022, pp. 148–160, doi: 10.1145/3558535.3559786.

[26] X. Qin et al., "Blindhub: Bitcoin-compatible privacy-preserving payment channel hubs supporting variable amounts," in *Proc. 44th IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2023, pp. 2462–2480, doi: 10.1109/SP46215.2023.10179427.

[27] V. Shoup, "Practical threshold signatures," in *Proc. Adv. Cryptology - EUROCRYPT Int. Conf. Theory Application Cryptographic Techn.*, Bruges, Belgium, B. Preneel, Ed., vol. 1807, 2000, pp. 207–220, doi: 10.1007/3-540-45539-6_15.

[28] M. Backes and D. Hofheinz, "How to break and repair a universally composable signature functionality," in *Proc. Inf. Secur. 7th Int. Conf.*, Palo Alto, CA, USA, K. Science Zhang and Y. Zheng, Eds., vol. 3225, 2004, pp. 61–72, doi: 10.1007/978-3-540-30144-8_6.

[29] L. Aumayr et al., "Generalized channels from limited blockchain scripts and adaptor signatures," in *Proc. Adv. Cryptology - ASIACRYPT- 27th Int. Conf. Theory Application Cryptology Inf. Secur.*, Singapore, M. Science Tibouchi and H. Wang, Eds., vol. 13091, 2021, pp. 635–664, doi: 10.1007/978-3-030-92075-3_22.

[30] M. Chase, M. Orrù, T. Perrin, and G. Zaverucha, "Proofs of discrete logarithm equality across groups," *Cryptol. ePrint Arch.*, 2022, Paper 2022/1593. [Online]. Available: https://eprint.iacr.org/2022/1593

[31] Z. Sui, J. K. Liu, J. Yu, M. H. Au, and J. Liu, "Auxchannel: Enabling efficient bi-directional channel for scriptless blockchains," in *Proc. ASIA CCS '22: ACM Asia Conf. Comput. Compat. Secur.*, Nagasaki, Japan, Y. Suga, K. Sakurai, X. Ding, and K. Sako, Eds., 2022, pp. 138–152, doi: 10.1145/3488932.352412.

[32] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," *IEEE Symp. Secur. Privacy (SP)*, pp. 315–334, 2018, doi: 10.1109/SP.2018.00020

[33] F. Boudot, "Efficient proofs that a committed number lies in an interval," in *Proc. Adv. Cryptology - EUROCRYPT Int. Conf. Theory Application Cryptographic Techn.*, Bruges, Belgium, B. Preneel, Ed., vol. 1807, 2000, pp. 431–444, doi: 0.1007/3-540-45539-6_31.

[34] A. D. Santis, S. Micali, and G. Persiano, "Non-interactive zero-knowledge proof systems," in *Proc. Adv. Cryptology Conf. Theory Appl. Cryptographic Techn.*, Santa Barbara, California, USA, C. Pomerance, Ed., 1987, vol. 293, pp. 52–72, doi: 10.1007/3-540-48184-2_5.

[35] A. C. Mert, E. Öztürk, and E. Savas, "Low-latency ASIC algorithms of modular squaring of large integers for VDF evaluation," *IEEE Trans. Comput.*, vol. 71, no. 1, pp. 107–120, Jan. 2022, doi: 10.1109/TC.2020.3043400.

[36] S. Saereesitthipitak and D. Zindros, "Cassiopeia: Practical on-chain witness encryption," in *Proc. Financial Cryptography Data Secur., Int. Workshops Voting, CoDecFin, DeFi, WTSC, Bol*, C. Brač, A. Science S. Essex O. Matsuo L. Kulyk A. Gudgeon Klages-Mundt, D. Perez, S. A. Werner Bracciali, and G. Goodell, Eds., 2023, vol. 13953, pp. 385–404, doi: 10.1007/978-3-031-48806-1_25.

[37] F. Benhamouda et al., "Can a public blockchain keep a secret?," in *Proc. Theory Cryptography 18th Int. Conf.*, TCC Durham, NC, USA, R. Pass and K. Pietrzak, Eds., vol. 12550, 2020, pp. 260–290, doi: 10.1007/978-3-030-64375-1_10.

[38] A. Poelstra, "Scriptless scripts," 2017. [Online]. Available: https://tinyurl.com/ludcxyz

[39] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei, "Anonymous multi-hop locks for blockchain scalability and interoperability," in *Proc. 26th Ann. Netw. Distrib. System Secur Symp.*, San Diego, California, USA, The Internet Society,2019. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/anonymous-multi-hop-locks-for-blockchain-scalability-and-interoperability/

[40] L. Eckey, S. Faust, K. Hostáková, and S. Roos, "Splitting payments locally while routing interdimensionally," *Cryptol. ePrint Arch.*, 2020, Paper 2020/555. [Online]. Available: https://eprint.iacr.org/2020/555

[41] A. Erwig, S. Faust, K. Hostáková, M. Maitra, and S. Riahi, "Two-party adaptor signatures from identification schemes," in *Public-Key Cryptography PKC 24th IACR Int. Conf. Practice Theory Public Key Cryptography*, J. A. Garay, Ed., 2021, vol. 12710, pp. 451–480, doi: 10.1007/978-3-030-75245-3_17.

[42] Y. Zhang, Y. Long, Z. Liu, Z. Liu, and D. Gu, "Z-channel: Scalable and efficient scheme in zerocash," *Comput. Secur.*, vol. 86, pp. 112–131, 2019, doi: 10.1016/j.cose.2019.05.012.

[43] L. Aumayr, S. A. K. Thyagarajan, G. Malavolta, P. Moreno-Sanchez, and M. Maffei, "Sleepy channels: Bi-directional payment channels without watchtowers," in *Proc. 2 ACM SIGSAC Conf. Comput. Commun. Secur.*, Los Angeles, CA, USA, H. A. Yin, C. Stavrou Cremers, and E. Shi, Eds., 2022, pp. 179–192, doi: 10.1145/3548606.3559370.

[44] Z. Sui, J. K. Liu, J. Yu, and X. Qin, "MoNet: A fast payment channel network for scriptless cryptocurrency Monero," in *Proc. 42nd IEEE Int. Conf. Distrib. Comput. Syst.*, Bologna, Italy, 2022, pp. 280–290, doi: 10.1109/ICDCS54860.2022.00035.

[45] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. 42nd Ann. Symp. Foundations Comput. Sci.*, Las Vegas, Nevada, USA, Oct. 2001, 2001, pp. 136–145, doi: 10.1109/SFCS.2001.959888.

[46] R. Canetti, Y. Dodis, R. Pass, and S. Walfish, "Universally composable security with global setup," in *Proc. Theory Cryptography, 4th Theory Cryptography Conf.*, Amsterdam, The Netherlands, S. P. Vadhan, Ed., 2007, vol. 4392, pp. 61–85, doi: 10.1007/978-3-540-70936-7_4.

Yuan Li is currently working toward the PhD degree with the College of Cyber Security, Jinan University, Guangzhou, China. His research interests include blockchain systems and applied cryptography, with a particular focus on secure protocol design, privacy-preserving mechanisms, and decentralized system security. His work explores cryptographic constructions and security analysis for trustworthy and dependable distributed systems.

Junzuo Lai received the PhD degree in computer science and technology from Shanghai Jiao Tong University, China, in 2010. From 2008 to 2013, he was a research fellow with Singapore Management University. He is currently a professor from the College of Cyber Security, Jinan University, China. His research interests include cryptography and information security. He has authored or couthored more than 40 papers in international conferences and journals such as *EUROCRYPT*, *ASIACRYPT*, *PKC*, *IEEE Transactions on Dependable and Secure Computing* and *IEEE Transactions on Information Forensics and Security*.

Yi Liu received the PhD degree in computer science from The University of Hong Kong in 2023. He is currently a lecturer with Jinan University, Guangzhou. His research interests include secure two-party/multi-party computation, zero-knowledge proofs, timed cryptography, blockchain-related applications. He has authored or coauthored several papers at international conferences, such as *ASIACRYPT*, *PKC*, *ESORICS*, and *ACNS*.

Liang Zhang received the bachelor's degree from Huazhong University of Science and Technology and the PhD degree from Fudan University in 2022. He was with Baidu Company, Ltd where he focuses on excellent programming skills. He is currently a postdoctoral fellow with Hong Kong University of Science and Technology. His publications are included in *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Computers*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Big Data*, *IEEE Transactions on Cloud Computing*, *CJ*, *CSCWD'25*, *APWeb-WAIM'23*, *CF'22,* and *SACMAT'22*. His research interests include blockchain and applied cryptography.

Haiyan Wang received the PhD degree in cryptography from the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China, in 2017. She is currently an associate research fellow with the Department of New Networks, Pengcheng Laboratory, Shenzhen, China. She has published several papers at international journals, such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, and *IEEE Transactions on Mobile Computing*. Her research interests include information network security and applied cryptography.

Ke Ma is currently working toward the MS degree with the College of Cyber Security, Jinan University, Guangzhou, China. His research interests include applied cryptography and network security, particularly in the design and analysis of secure protocols.

Jiheng Zhang was with IEDA as an assistant professor in 2009, and was promoted to associate professor in 2015 and to full professor in 2020. His research interests include the areas of service operations management, queueing system and applied probability. He is an outstanding scholar in the field of Stochastic Modelling and Optimization, Statistical Learning, Numerical Methods and Algorithm, as evidenced by publishing ten papers in leading journals of his discipline since his substantiation, including Management Science, Operations Research, and Mathematics of Operations Research. He is an associate editor for *Operations Research*, *Stochastic Systems*, and *Queueing Models and Service Management*, and an editorial board member for *Probability in the Engineering and Informational Sciences*.